

区块链基础 - Lecture 6

6 部分同步模型、33%和CAP定理 - The Partially Synchronous Model and the CAP Principle

Lecture 6 原版讲义链接: <https://timroughgarden.github.io/fob21/1/l6.pdf>

Lecture 6 原版PPT链接: <https://timroughgarden.github.io/fob21/slides/F21L6.pdf>

6.1 半同步模型 - The Partially Synchronous Model

6.1.1 概要 - Overview

本章有三个目标。

一、介绍部分同步计算模型，它是同步模型和异步模型的折中方案。如果你只想记住一个模型，那么就记住这个模型。部分同步模型是工程上的甜区模型，因为它的假设既足够弱，能够适用于互联网规模的分布式协议设计，又足够强，能够使得一些具有可证明性质的协议存在。

二、明确我们对部分同步模型期望的限制。具体来说，我们将会证明一个不可能性结果，当拜占庭节点占比大于等于三分之一时（ $f \geq \frac{n}{3}$ ），就无法得到可证明的好结果。我们在区块链项目白皮书或技术讨论中经常看到的33%这个阈值，就是指这个不可能性结果。这个证明不会太复杂，没有FLP那么晦涩难懂。学习这个证明的好处是，你可以记得住它背后的直觉。

三、讨论分布式系统中的一个著名定理，即CAP定理（The CAP Principle / Theorem）。C表示Consistency，即一致性；A表示Availability，即可用性；P表示Partition-tolerance，即分区容错性。CAP定理指出，你只能在这三者之中取其二，无法三者兼得。CAP定理与异步模型中的FLP定理很相似，因此理解它们之间的区别很重要，不可混淆。二者虽然都揭示了分布式系统的局限性，但本质假设和结论不同。听完这节课后，若有人使用CAP定理为他

们系统的缺点进行辩解，你应该能够判断他们是否真的知道自己在说什么。

6.1.2 回顾同步模型和异步模型 - Recap: Synchronous vs. Asynchronous Model

为了引出部分同步模型（partially synchronous model），我们先来回顾一下之前学习过的同步模型和异步模型。

同步模型（synchronous model）：

- 共享的全局时钟，所有节点无需通信就知道当前是哪个时间步
- 消息延迟有一个先验已知的上界 Δ ，任何消息在第 t 时间步被投递，最迟会在第 $t + \Delta$ 时间步送达
- 优点：可以实现可证明的好结果，例如在PKI假设下，Dolev-Strong与其派生的SMR协议（例如结合Round-Robin leader）可以容忍99%的拜占庭节点
- 缺点：假设不现实，现实中的互联网做不到消息延迟不超过 Δ ，例如因断电造成消息丢失，因攻击造成不可预知的消息延迟等

异步模型（asynchronous model）：

- 没有全局时钟
- 除了消息最终可达性约束外，对消息延迟不做任何假设，消息可能无限延迟但不会永远丢失
- 优点：在这么弱的假设下，任何一个正向的结果都有极高的理论价值
- 缺点：根据FLP定理，即使只有一个崩溃故障，任何确定性协议都无法解决拜占庭共识问题或SMR问题

无论是同步模型还是异步模型，都没有让我们实现我们想要实现的目标，一种在互联网规模下可能实用的共识协议。因此，我们显然需要第三种模型，它是这两种模型的一个折中方案，假设要足够弱，以适配互联网规模的系统，也要足够强，使得可证明正确的可实践的共识协议成为可能。

思考：同步模型假设排除了网络中有服务中断和遭遇攻击的现实，而异步模型假设网络可能遭受持续不断的攻击。现实中的互联网更有可能是处于二者之间的一种状态，即网络的服务中断和攻击极有可能是一种暂时的状态，中断和攻击总会在某个时刻停下来。那么网络便可以在服务中断或遭受攻击时采用异步模型的策略，而在稳定时采用同步模型的策略，这样就能对consistency和liveness有一定的保障，这便是半同步模型。

6.1.3 半同步模型描述

直觉：系统总有一天会从攻击和服务中断中恢复

在互联网中，系统总有可能会遭受不可预知的攻击，且攻击和服务中断持续的时间也是无法预测的，这让我们无法接受同步模型。不过，攻击和服务中断必然会在某个时刻终止（或许持续几个小时，亦或是几天），即系统在某一个时刻必然会恢复为“正常”状态。

因此，我们不妨按此将系统的运行时间分为“正常”时段（建模为同步模型）和“遭受攻击”时段（近似建模为异步模型）。在这样的模型下，我们的目标可以是：

- 在系统运行的正常时段，获得我们想要的性质，例如安全性和活性
- 在系统运行的遭受攻击时段，不要让系统完全崩溃，例如至少保留一种性质
- 当攻击结束后，系统能够快速恢复为正常状态

半同步模型当然可以建模为在n个同步阶段和异步阶段之间不停地转换。为了方便研究，我们采用一个最小模型，即建模为只有一个同步阶段和一个异步阶段。更为复杂的半同步模型可以通过这个最小模型进行链式扩展而得。为了研究系统如何从攻击中恢复，我们可以假设系统首先处于异步阶段，然后转换为同步阶段。

形式化定义 - The Formal Definition

半同步模型的形式化定义是由Dwork, Lynch和Stockmeyer在论文[Consensus in the presence of partial synchrony](#)中完成，他们也对一系列分布式系统中的可能性和不可能性结果进行了证明，做出了非凡的贡献。

时间假设：与同步模型类似，第一个假设是系统中存在一个共享的全局时钟。即，无论系统处于受攻击时段还是正常时段，所有节点在不通信的情况下都对当前的时间步达成一致。显然我们不想做出该假设，但在同步模型中的两个假设里，这无疑是更容易接受的假设。全局时钟假设可以做适当的放宽，允许不同的节点之间存在时钟的有界漂移（bounded drift）。

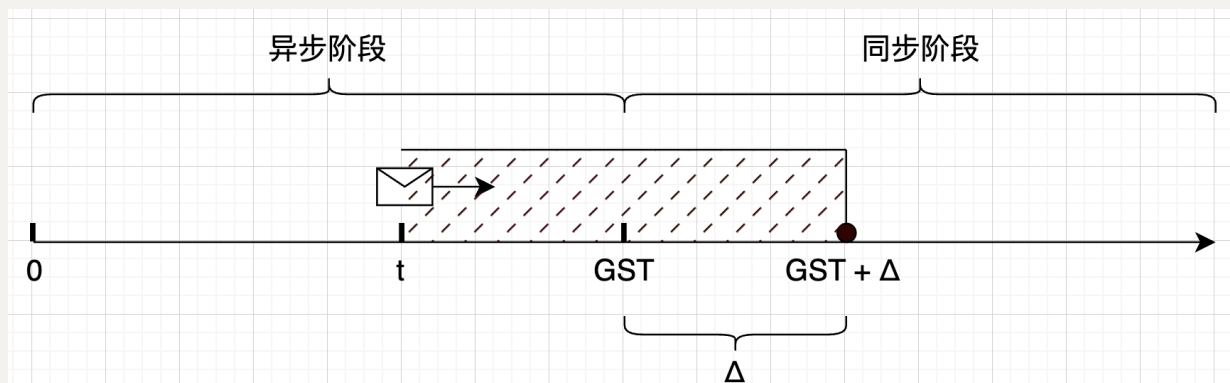
同步阶段：第二个假设，是存在一个参数 Δ 为一条消息在同步阶段中的最大延迟（以时间步为单位）。在初始异步阶段中，参数 Δ 不起作用。与同步模型类似，在半同步模型中，我们假设 Δ 的值是一开始就已知的，协议定义是基于这个值的。例如，你可以把1个时间步看成是1毫秒， Δ 的值可以是1000或2000个时间步，即1秒或2秒。

GST (Global Stabilization Time, 全局稳定时间)：最后一个假设是 GST ，该参数决定网络通信何时从异步模式转换为同步模式（讲义原文是"...switches from synchronous to asynchronous mode..."，应该是教授写反了）。与 Δ 不同， GST 的值不是一开始就知道的，因此协议定义不能依赖该值。也就是说，无论 GST 何时发生，协议都要对其承诺的保证负责。没有人会通知协议 GST 是否已经发生，协议必须自行检测模式转换是否已经发生，并根据检测结果恢复正常运行。

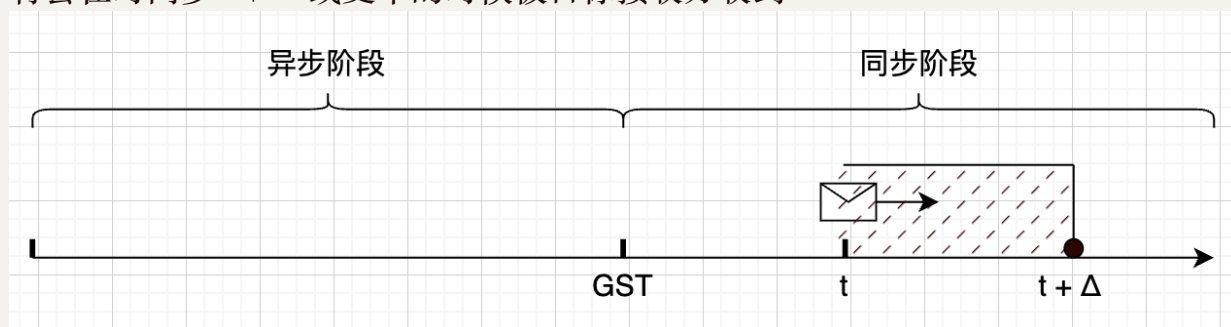
注： GST 在这里是一个理论上的时刻/时间点，指的是系统从异步通信模式转换成同步通信模式的那个时刻， GST 的定义本身不依赖节点行为和测量。在实践上，协议并不知道这个时刻什么时候发生，也没有一个标准来衡量这个时刻的发生，只能通过经验性的metrics去判断 GST 是否已经发生。比如某个节点在连续多轮内都能按时收到所有其他节点的消息，那么可以判定 GST 发生；或者通过心跳+超时来判断，如果超时次数明显减少，达到一个经验值或者用统计学方法判断超时在某个“稳定区间”内，那么可以判定为 GST 已经发生。

综合来看，参数 GST 和 Δ 对消息投递强制施加了如下约束（其中 Δ 是先验已知的；而 GST 值是由消息投递敌手选择的，是未知的，且无论敌手恰好选择了什么样的值，约束都存在）：

1. （异步阶段发送的消息）如果一个消息在时间步 t 发送，且 $t \leq GST$ ，那么该消息将会在时间步 $GST + \Delta$ 或更早的时候被目标接收方收到（即，可以视作消息是在 GST 时刻被发送的）



2. (同步阶段发送的消息) 如果一个消息在时间步 t 发送, 且 $t \geq GST$, 那么该消息将会在时间步 $t + \Delta$ 或更早的时候被目标接收方收到



注: $t < GST$ 时, 系统处于异步阶段, $t \geq GST$ 时, 系统处于同步阶段。如果消息是在 $t = GST$ 时发出的, 那么消息最晚会在 $GST + \Delta = t + \Delta$ 收到。

由于 GST 必须是有限的, 因此所有消息最终必然在有限时间内被送达。从这点来看, 半同步模型其实是异步模型的一个特例, 消息在异步模型和半同步模型都会最终被送达, 只是前者可能是无限长时间, 后者是有限长时间。因此异步模型是严格意义上更一般化的模型, 因为消息送达时间没有定量的约束。

关键在于, GST 的值对协议来说是未知的。控制消息传递的敌手可以根据协议的行为来“选择” GST 的具体时间点。也就是说, 敌手可以根据协议的设计来延迟 GST , 逼迫系统暴露安全性或活性问题。因此, 在协议设计的时候若要提升健壮性, 则设计不可依赖 GST 的具体值。

之所以半同步模型在建模的时候让 GST 保持未知, 是因为该模型的核心目标就是迫使我们应对现实中不可预知时长的网络中断和攻击。如果 GST 是已知的, 那么协议设计可以偷懒, 依赖已知的 GST , 这样设计出来的协议难以应对真实的网络状况。另一个理由是, 如果 GST 是已知的, 那么同步模型中所有可能性结果可以直接照搬过来, 协议只需要等到 GST 之后再按照同步模型的协议设计来执行即可, 那为何不直接用同步模型呢。

协议: 后续我们将证明半同步模型的一个不可能性结果, 因为我们需要明确定义协议的能力边界, 即协议能够做什么, 不能够做什么。在异步模型中, 协议定义了节点的行为 (即选择发送哪些消息), 这是一个基于节点当前已知信息的函数, 这些已知信息包括私有输入

(private input) 和到此刻为止节点收到的所有消息之序列。由于半同步模型引入了时间假设, 节点的已知信息增加了一些, 包括此刻的时间步、每条消息具体在哪个时间步收到的。那么在半同步模型中, 一个协议可以定义为基于节点目前已知信息映射到其行为的函数。已知信息包括私有输入、当前的时间步、收到的消息序列以及消息到达的时间步, 行为则是选择哪些消息发送以及发送的目的地。与异步模型不同的是, 半同步模型中的协议可以实现超时机制 (例如忽略那些比预期到达时间晚了很多个时间步的消息)。后续课程我们会讲解Tendermint协议如何利用这一点。

未知的 Δ 模型：半同步模型其实有2个不同的版本，各位或许对此会有点疑惑（这2个版本都是Dwork、Lynch和Stock Meyer定义的）。在本系列课程中，我们只采用GST版本的模型。但在其他的地方，你可能会遇到另一个版本的模型，即未知的 Δ 模型。我们在这里对这个模型做一个简要的介绍。

第二个版本的模型与第四章开头的讨论密切相关。在第四章，我们对同步模型的假设放宽做过粗略的尝试（假设 $\Delta = 1$ ），将其扩展为一个消息延迟可变的模型（延迟在1到一个已知的上限 Δ 之间）。随后我们注意到，任意一个对 $\Delta = 1$ 的模型成立的可能性结果，都可以通过time inflation（时间膨胀）自动推广为一个更一般的情况（ Δ 是已知的），即所有节点在每次操作之间都固定等待 Δ 个时间步，以确保来自诚实节点的消息都能及时送达。对此我们并不满意，我们期望的是一个运行速度不受最坏的消息延迟限制，并且能够自适应网速的共识协议。如果底层的通信网络速度是足够快的，那么协议本身也应该匹配这样的网速。这就是第二个版本的半同步模型背后的基本思想。

在未知 Δ 版本的模型中，没有GST假设，也就是没有从异步阶段到同步阶段转换的过程。所有消息都会在 Δ 个时间步被送达，这点和异步模型一样。与同步模型（以及GST版本的半同步模型）不同的是，这里的参数 Δ 并不是已知的，因此协议的定义不能依赖 Δ 。即，无论网速和 Δ 的值是多少，协议必须在所有同步模型的具体实现中都能正确工作。

为什么存在两个版本：这两个版本的半同步模型看起来都很自然。GST版本的建模是以系统宕机或遭受攻击后快速恢复为目标。而未知 Δ 版本的目标是系统根据任意的网速能够自适应运行。之所以它们都被称为半同步模型，是因为提出partial synchrony概念的原始论文就是将这两个版本都放到了半同步这个概念下。另一个原因是，在其中一个版本的模型中成立的可能性或不可能性结果，通常在另一个版本的模型中也成立。因此，就我们的研究目的而言，这两个版本在效果上大致等价，但形式上不同。作为练习，读者可以尝试在未知 Δ 的版本下重新构造FLP定理的证明。你当然也可以思考如何改造Tendermint协议，使一致性和活性（ $f \leq \frac{n}{3}$ ）在未知 Δ 版本的模型中也成立。

本系列课程后续不再讨论未知 Δ 版本的模型。

6.1.4 共识协议的目标 - Goals for a Consensus Protocol

在半同步模型中，我们应该对共识协议提出什么样的要求？例如，达到什么标准或要求后，我们才能认为一个协议在半同步模型中解决了BA问题？回顾第四章的内容，BA问题指的是，对任意节点 i 有私有输入 $v_i^* \in V_i$ ，并且BA协议应当有如下性质：

1. 终止性（Termination）。每个诚实节点 i 最终都会输出 $w_i \in V$ 并停止运行；

2. 一致性 (Agreement)。无论私有输入为何，所有诚实节点都会输出同样的值并停止运行；
3. 有效性 (Validity)。对任意节点 i ，若 $v_i = v^*$ ，则 $w_i = v^*$ 。

FLP定理告诉我们，在异步模型中，任何一种确定性的共识协议都无法同时满足这三个性质。半同步模型由于对消息投递施加了限制，而异步模型更为一般化，因此FLP定理无法直接适用于半同步模型（在第七章中，我们会学习一个可能性结果，用以说明FLP定理在半同步模型中为何不成立。）。不过我们知道半同步模型中有异步阶段，这个阶段是适用FLP定理的。因此，任何保证在终止时满足有效性和一致性的协议，必然会出现在GST之前无法终止的情况。也就是说，如果协议必须始终满足有效性和一致性，即不容忍安全性错误，那么它就无法在异步阶段保证活性，即一定会终止。同理，对于SMR问题而言，任何始终保证一致性的协议在GST之前也无法保证活性。然而在GST之后，协议就有可能同时实现这两个性质了。在GST后，我们不能仅通过切换到某种适用于同步模型的协议来保证安全性和活性。因为GST是未知的，是可以由敌手控制的，节点无从得知何时切换协议。在设计协议时就要考虑让协议自适应网络变化。

总体而言，我们所能期望的最佳情况大概是：

- 在异步阶段，即GST之前，至少能保证安全性或活性中的一个性质；
- 在同步阶段，即GST之后，同时保证安全性和活性。

那么在异步阶段我们要放弃什么？最自然的选择，也是20世纪关于半同步模型的研究中唯一被探讨过的选择，就是放弃活性，保留安全性。

安全性指的是“坏事永远不会发生”，其中的“永远”可以理解为即便在异步阶段也不应该出错。

活性指的是“好事最终会发生”，其中的“最终”或许可以认为是在GST之后。

许多区块链共识协议采用的正是这两种目标，即始终保证安全性，以及在GST之后实现最终活性，这也是本章和下一章的重点。

不过有一点很有意思，最长链共识机制（第八章）在异步阶段做出了非传统的权衡，即更偏向活性而非安全性。

6.1.5 一个重要定理

在半同步模型中，比如BA问题，在什么条件下才能实现安全性和最终活性（eventual liveness）？这个问题有一个非常明确简洁的答案，也是读者可能在区块链领域经常看到的一个数字，一个神奇的阈值，**33%**。

定理6.1： 在半同步模型中，当且仅当 $f \leq \frac{n}{3}$ 时，存在一个确定性拜占庭协议能同时满足一致性、有效性和最终活性（GST之后）。

这个定理中实际上包含两个结果，一个可能性结果和一个与之相对的不可能性结果。我们着重讲一下不可能性结果，即当 $f \geq \frac{n}{3}$ 时，不存在任何一种拜占庭协议能够同时满足安全性和最终活性。

在第七章中我们会学习Tendermint协议，该协议实际上就是在 $f < \frac{n}{3}$ 时保证了安全性和最终活性。下面是一些注释：

1. Tendermint只是众多在半同步模型下实现最优容错能力的协议之一（比如在原始论文中还有另一种解决方案）。由于Tendermint协议已经被多个主流的区块链协议所采用，因此我们会重点介绍该协议。
2. 定理6.1不依赖于PKI假设。本章我们将要证明的不可能性结果在PKI假设下依然成立，因为拜占庭节点不需要伪造签名也能破坏系统。Tendermint协议是依赖PKI假设的，但同样的正向结果在没有PKI的情况下在另一个协议中也能实现。
3. 与FLP定理一样，这个不可能性结果虽然是针对拜占庭一致性问题提出的，但同样适用于SMR问题。第七章中介绍的Tendermint协议解决的是SMR问题，我们可以通过归约方法构造出一个具备相同容错性的BA协议。
4. 你可以发现分布式计算专家经常把 $f \leq \frac{n}{3}$ 写成等价的形式 $n \geq 3f + 1$ 。这个不等式在分布式计算领域变成了一个梗……（理工男冷梗，不用管它）
5. 每当你听到链圈人或白皮书提到33%这个数字，通常是为了说明协议为什么无法应对更强的攻击，其实就是在引用定理6.1。这是半同步模型下拜占庭协议的理论容错极限，超过这个阈值就不可能保证安全性和活性。
6. 如果你只能从2-7章中记住一个定理，那就记住6.1。它是我们对20世纪共识协议设计和分析研究的总结性成果。

6.2 神奇数字33%之不可能性定理的直觉

本节的内容主要是给读者提供一种合理的、准确的直觉解释，说明为什么定理6.1之不可能性结果中的“当且仅当”是正确的。这个直觉性的论证会揭示这个神秘的33%阈值是从哪儿来的，而回过头来看，这个数字会显得像是命中注定的。33%这个数字不是偶然的，而是不可避免的结构性的结果。这个非正式的论证非常简洁，你应该能够在脑海中记住很长一段时间。

消息延迟无上限的作用 首先，我们讨论一下证明的策略，以及我们期望在论证中想要看到的关键要素。我们在第三章的PSL-FLM不可能性结果在同步模型且无PKI假设的BB问题（Byzantine Broadcast，拜占庭广播）确立了33%这个阈值，因此该阈值并不是在定理6.1中第一次看到。

那么，这些33%的含义是相同的吗？

有没有可能定理6.1可以用某种方式归约到我们曾经费尽心力所证明的PSL-FLM的不可能性结果？也许在容错能力的程度上，同步模型和半同步模型其实没有本质的区别？

实际上，这两个不可能性结果的背后是完全不同的驱动因素。要记得的是，PSL-FLM不可能性结果在PKI假设下不成立。在PKI假设下，Dolev-Strong协议解决了BB问题，即便有99%的节点是拜占庭的，该协议也能够满足一致性、有效性和终止性。因此，PSL-FLM的不可能性结果的证明之关键在于其依赖于PKI不存在。这种依赖以一种微妙的方式体现出来，证明中使用了很巧妙的策略构造了拜占庭节点，通过模拟其余四个诚实节点的行为欺骗另外两个节点（六边形思想实验）。这种策略在PKI假设存在的情况下是不可行的，因为它们需要在没有对应私钥的情况下伪造签名，而PKI假设中包含了伪造签名是不可能的假设。

在上一节我们提到过，无论我们是否假设PKI存在，定理6.1都成立。故，第八章中对定理6.1的证明应该不会用到用拜占庭节点模拟诚实节点的策略。而在PSL-FLM的不可能性证明中，这个策略是唯一的一个技巧，因此定理6.1中的33%阈值必然是由其他因素造成的。

考虑到定理6.1的不可能性结果是针对半同步模型的，我们可以推测，这个阈值可能是消息延迟无上限这个潜在威胁带来的。

注：在GST之前，消息延迟可以无穷大。敌手可以利用这一点制造网络不确定性，诱导诚实节点在异步阶段做出不同的决策，从而破坏安全性

直觉1：可合理否认的冷处理策略。首先，哪怕在同步模型中，为了满足终止性，诚实节点即使没有从任何节点收到消息，也必须输出值并终止。拜占庭节点的一种策略就是对诚实节点进行冷处理（装死），即完全不发送任何消息。由于至多有 f 个节点是拜占庭的，因此诚实节点只能期望从至多 $n - f$ 个节点那里收到消息，从而做出决策（如果一个诚实节点已经收到了 $n - f$ 个节点的消息，那么它不用一直等下去，因为拜占庭节点有可能永远不会发出消息）。

直觉2：披着羊皮的狼。由于我们现在讨论的是半同步模型，因此诚实节点若没有收到某些节点的消息，对此也可以有一个同样合理的解释。有可能那些节点实际上是诚实节点，确实发送了所有该发送的消息，只是这些消息被严重延迟了。当然，这种情况只可能发生在GST之前，但GST在什么时候发生是协议不知道的，异步阶段可能持续很长时间。在同步模型中，就不会发生这种事。如果你没有收到某个节点该发送的消息，我们可以断定该节点是拜占庭节点。因此，比较麻烦的是，当一个诚实节点在收到 $n - f$ 个节点的消息后，它必须进行决策。但是，该诚实节点并不知道，有可能这些 $n - f$ 个节点里头，有 f 个实际上是拜占庭节点，而其余节点可能是诚实的，它们发送了消息但被严重延迟，从而导致该节点没有收到这些消息。诚实节点被迫基于有限的信息做出决策，而它手头的消息有可能主要来自拜占庭节点。就像披着羊皮的狼一样，这些拜占庭节点可能通过给诚实节点提供错误的信息来误导诚实节点做出决策。比如，拜占庭节点可以假装自己的私有输入是1而不是0。从直觉上看，如果这 $n - f$ 个节点中，至少有一半是拜占庭的，那么诚实节点就无法判断该相信谁。若没有诚实多数节点，任何基于多数投票的机制都可能被拜占庭节点操纵。如果这些节点中有严格多数是诚实的，那么我们就可以寄希望于某种多数投票机制来指引诚实节点做出正确的决策。于是我们可以推断出， $f < \frac{n-f}{2} \Leftrightarrow f < \frac{n}{3}$ 。

梳理一下神奇阈值33%的推导逻辑：

1. 由于诚实节点必须满足终止性，且拜占庭节点可能永远不会发送任何消息，故一个诚实节点在做出决策前，至多等待 $n - f$ 个节点（包括自己）发送过来的消息；
2. 诚实节点为了防止被欺骗，这 $n - f$ 个节点中必须有严格多数个节点是诚实的，故 $f < \frac{n-f}{2}$ ，即 $f < \frac{n}{3}$ 。

这个非正式的论证揭示了一个关键难点，拜占庭节点与敌手操纵消息投递或形成共谋。拜占庭节点和操纵消息投递的敌手都有能力让 f 个节点长期保持沉默——拜占庭节点可以故意不发送消息，操纵消息投递的敌手可以拖延诚实节点投递的消息，而在诚实节点视角看，结果都是没有收到某些节点的消息，这让诚实节点无法区分那些节点究竟是拜占庭的，还是诚实的（但消息被敌手拖延了，比如可以想象GFW是这样的一个敌手）。

实际上，每个敌手有能力挫败2个诚实节点：一种方式是刚才提到的，延迟诚实节点投递的消息，让其他的诚实节点误以为这些节点是拜占庭节点；另一种方式是拜占庭节点发送与某个诚实节点相冲突的消息来抵消该诚实节点的信息对其余诚实节点的决策作用。

上一段中的“敌手”包括操纵消息投递的敌手和拜占庭节点，这里我们把敌手的概念泛化为一切有能力对系统造成破坏的对手。有时候我们会混用拜占庭节点和敌手这两个术语，在建模所有类型的攻击对系统造成总体破坏的时候，这两个术语是等价的，在讨论不同类型的破坏策略的时候，这两个术语是有区别的。

我们希望这2个诚实节点被挫败后，至少还剩下一个能正常运行的诚实节点，也就是 $(n - f) - 2f > 0$ ，即 $f < \frac{n}{3}$ 。

其中， $n - f$ 表示诚实节点在决策前至多能等待 $n - f$ 个节点发送过来的消息，也就是有 $n - f$ 个节点或许是诚实的；（ f 是模型假设的已知的拜占庭节点的数量，当然是要扣除的）

由于每个拜占庭节点（这里拜占庭节点的概念等价于敌手）即敌手能让2个诚实节点失效，也就是在 $n - f$ 个节点里面，其中有 $2f$ 个诚实节点可能是失效的（ $n - f$ 是扣除了已知的拜占庭节点的数量，还剩下的节点有可能是诚实节点，但这些节点中，有 $2f$ 个节点可能会被攻击从而失效，也需要扣除）；

我们当然期望拜占庭节点数量和可能会被攻击的诚实节点数量全都扣除后，剩下还能正常运行的诚实节点数量是大于0的，因此得到 $(n - f) - 2f > 0$ 。

上述是直觉上的解释，我们需要更为严谨的能够令人信服的形式化证明，否则作为一个定理是不严谨的。

6.2.1 定理6.1之不可能性结果的证明

不可能性命题陈述：在半同步模型中，无论是否存在PKI假设，对任意 f 和 n ，只要 $f \geq \frac{n}{3}$ ，则不存在任何拜占庭协议能同时满足一致性、有效性和最终终止性。

一个简单但包含所有复杂性的特例：

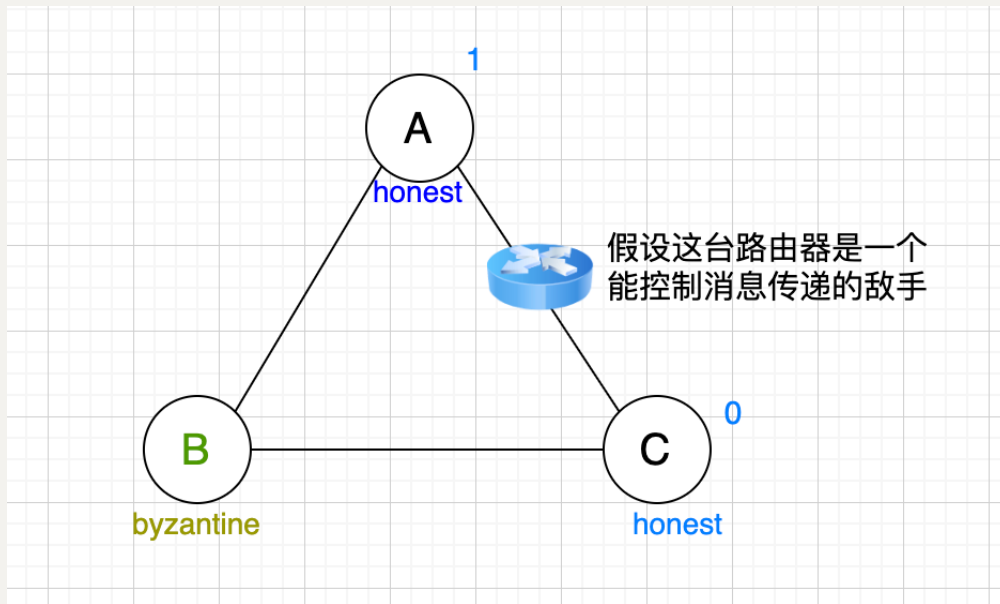
我们尝试从最简单的情况开始，即 $n = 3, f = 1$ 。一共有3个节点，其中至多有一个可能是拜占庭节点。

不要觉得这个特例很简单，似乎把不可能性结果的证明平凡化了，读者可以参考PSL-FLM不可能性结果的证明，将下面的论证进行推广并一般化。

证明策略依然是使用反证法。

设 $n = 3, f = 1, V = \{0, 1\}$ 。即有三个节点，Alice (A)，Bob (B)，Carol (C)，其中B为拜占庭节点。假设存在一个确定性的拜占庭协议 π 能够在半同步模型中同时满足一致性、有效性和最终终止性。

考虑下述场景：Alice是诚实节点，其私有输入为1；Carol是诚实节点，其私有输入为0；Bob是拜占庭节点，因此不用假设其私有输入。



设想B节点采用典型的拜占庭策略，向不同的诚实节点发送不一致的消息，同时又获得了能控制消息投递敌手（那台该死的路由器）的配合，敌手故意拖延A节点和C节点之间的所有消息（在GST之前，消息可以无限延迟）。

B节点在与A节点交互的过程中，假装自己的私有输入是1，并且假装自己从没有收到过C节点的消息；B节点在与C节点交互的过程中，假装自己的私有输入是0，并且假装自己从没有收到过C节点的消息。无论 π 协议是否存在PKI假设，B节点都能通过同时运行两个 π 协议副本来达到上述目的。

由于在半同步模型中，GST可以大到任意程度，所以敌手有能力任意延长A和C之间的消息投递。

两难困境：

从A节点视角看，B节点一直在投递消息给我（有可能是假消息），而C节点一直沉默。A节点很清楚存在这样一种可能：B节点是拜占庭节点，它在用上述策略伪装，而C节点的消息被敌手拖延了。然而，也有可能是，B节点是诚实节点，私有输入就是1，而C节点是拜占庭节点，对A和B保持沉默。为了防止出现这种可能性，由于 π 协议满足最终终止性，无论A节点有没有收到C节点的消息，A节点必须在某个有限时间 T_1 之前做出决策。又由于 π 协议满足有效性，因此A节点的输出必须为1。

从C节点的视角看，C节点也陷入了同样的困境。同理，C节点也必须在某个有限时间 T_2 之前做出决策，且必须输出0。

完成证明：

假设敌手将A和C之间的所有消息延迟到 $\max(T_1, T_2) + 1$ （这对敌手来说是可行的，只要它把GST设置得比这个时间更晚即可），这样A和C在做出决策之前就不可能收到对方的消息。

如此一来，A节点会输出1，C节点会输出0。这与 π 协议满足一致性的假设矛盾。□

6.3 CAP原则 - The CAP Principle

注：CAP定理最初是在1999年以CAP principle被发表，在2000年的时候以布鲁尔猜想的形式于PODC会议提出。2002年，MIT的Seth Gilbert和Nancy Lynch已发表了对布鲁尔猜想的证明，从而正式成为定理。但是从数学角度看，这个证明的形式内容略显单薄，或许还不够严谨，因此是否配得上Theorem（定理）还有争议。故，这里我们依然称其为CAP原则而不是定理。

Gilbert, Seth; Lynch, Nancy (2002). "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services". *ACM SIGACT News*. **33** (2). Association for Computing Machinery (ACM): 51–59.

CAP原则解释

CAP原则是分布式系统中一个著名的观察性结论。CAP并不论文作者的首字母组合，而是我们期望分布式系统能够满足的三个性质的首字母组合。

我们先解释CAP：

- C表示consistency，即一致性（不是agreement）。它与SMR中的consistency相同。一致性要求数据库的副本集之间的数据在同一时刻全都保持一致，我们期望用户无论被路由到哪个副本上，执行同一条查询都能获得一样的结果。更一般的，一致性指的是用户在中心化系统和分布式系统上的使用体验是一致的。
- A表示availability，即可用性。类似SMR的liveness，活性。可用性表示，当一个用户在数据库中插入一条数据后，这条数据最终会被写入数据库并且能被查询到。更一般的，可用性指的是客户端在分布式系统中执行的任意命令最终都能被有效执行。

- P表示partition tolerance，即分区容错性。分区指的是网络中两组节点之间的通信长时间中断，比如网络遭受长时间的DDoS攻击。分区容错的意思是，即便出现了网络分区，我们也希望系统依然能够保持一致性和可用性。该性质在概念上与我们定义的半同步模型的异步阶段有些类似：在异步阶段，消息投递延迟可以是任意长的，从诚实节点角度看，这样的延迟与网络通信中断没有区别。但是，要注意二者在建模假设和行为上仍存在重要差异。

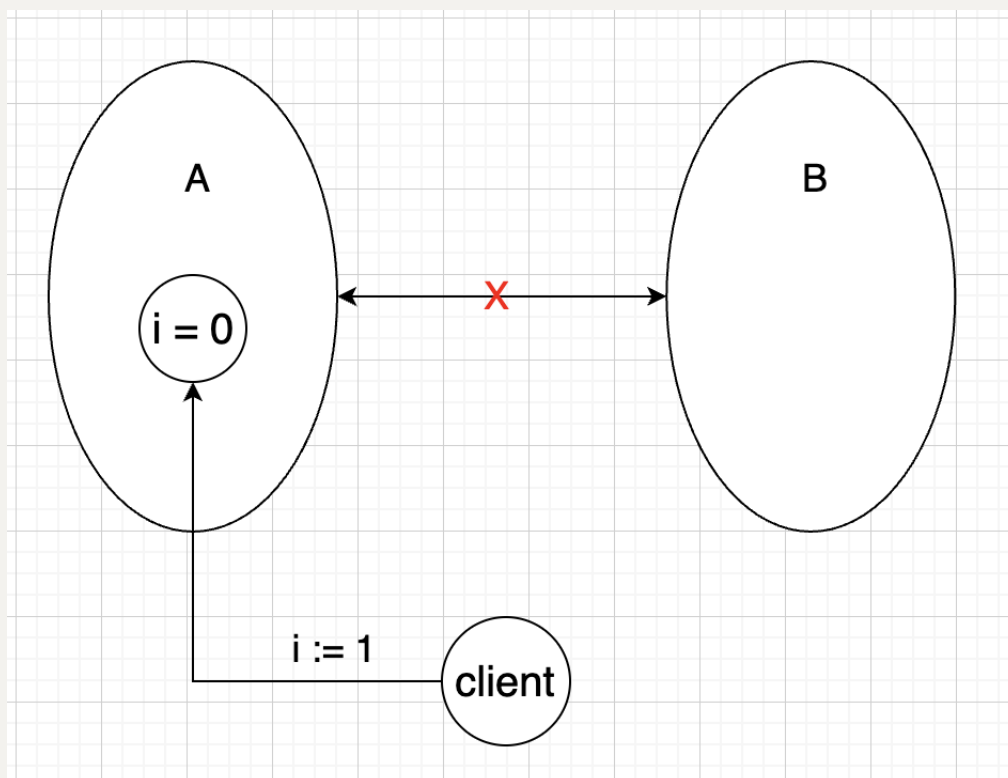
CAP原则断言：不存在任何分布式系统能够同时满足以下三种性质：一致性、可用性和分区容错性。

论证过程

CAP原则的论证其实很直观。试想一个分布式系统负责维护某个变量*i*的值，此时系统遭遇了网络分区，将网络分成了A组和B组。*i*的初始值是0，此时有一个客户端要更新*i*的值为1，客户端连接到了分组A的副本集中并执行了更新命令。

如果A组的副本集执行更新，由于A组和B组的通信中断，那么客户端在A和B中执行查询势必得到不一样的结果，违反一致性。

如果A组的副本集不执行更新，则违反可用性。



设计决策树

上述论证虽然看起来很平淡乏味，但它确实揭示出一种在设计分布式系统时很实用的决策路径：

- 判断你是否需要考虑网络分区，比如系统在开放的互联网环境里运行，就必须考虑，而系统部署在一个安全的私有网络中，则无需考虑；
- 如果不需要考虑，则可以同时要求一致性和可用性，这类似我们在同步模型下对SMR协议同时要求一致性和活性的做法；
- 如果必须考虑，那么需要评估应用场景，在一致性和可用性中做出权衡，如果出现了网络分区，牺牲哪个性质是可以接受的；

对于不同的应用场景，我们的决策是不同的。比如：

- 像银行这样的系统更倾向于牺牲可用性来保证一致性。这正是传统数据库做出的折中方案，相比于系统宕机24小时，账户余额不一致造成的经济损失是更无法忍受的；
- 类似搜索引擎这样的系统，往往更注重可用性，而一致性要求可以放宽一些（比如一些NoSQL数据库所采用的设计），在不同地理位置的用户看到的搜索结果会略有差异，但这点是相对可接受的，而宕机造成的损失则更大。

我们回到区块链协议设计中来，我们在接下来的两讲中看到类似的二选一：

- 一类SMR协议在受到攻击（半同步模型的异步阶段）时选择牺牲活性，例如Tendermint协议；
- 另一类协议更倾向于牺牲一致性，选择维持系统活性，例如最长链共识。

6.3.1 FLP不可能性定理 V.S. CAP原则

CAP原则的结论与FLP定理很类似，在遭受网络攻击（异步阶段/网络分区）时，你必须在安全性/一致性和活性/可用性之间做出选择。奇怪的是，你应该不太可能在同一门课程中同时学到FLP定理和CAP原则，前者是分布式计算理论课程里的内容，后者是分布式系统设计的内容。一个偏形式理论，一个偏系统工程。但任何区块链专家都应该清楚这两个定理/原则，并理解它们之间的区别。FLP定理的证明是困难的，而CAP原则的推理很简单。

CAP推理之所以简单，是因为它的设置允许两个分区的信息被无限延迟甚至丢弃，没有最终可达性约束，更没有GST。也就是说，在CAP中，控制信息传递的敌手的能力比FLP更强，这种额外的能力使得CAP的不可能性结论更容易推理。FLP不可能性定理则表明，即便敌手被强制要求最终必须投递所有消息，安全性和活性依然不可兼得。而在敌手能力相对较弱的前提下，证明这种不可能性就要困难得多。但从另一个维度上看，FLP模型中的敌手其实比CAP更强。在CAP中，敌手只能制造网络分区，而在FLP中，敌手可以做任何事情。但话又说回来，网络分区是一种经典的攻击，它已经能够涵盖大多数敌手的攻击策略。在CAP中，控制信息投递的敌手过于强大以至于我们无需再假设其他敌手的存在。换句话说，即便系统中所有节点都是诚实的，即 $f = 0$ ，CAP原则依然成立。相比之下，FLP定理在 $f = 0$ 的情况下就无法成立了，FLP需要至少一个故障节点才能成立。

再换句话说（感觉Prof.有时候废话有点多...）FLP真正揭示的是，哪怕存在只有一个节点宕机的威胁，也足以触发与无限延迟相同的结论，即在系统遭受攻击时，你必须在安全性和活性之间二选一。